

Porównanie działania wybranych algorytmów minimalizacji stochastycznej

Robert Mesek

Biblioteki

```
suppressMessages(library(ecl))
suppressMessages(library(smoof))
suppressMessages(library(plot3D))
suppressMessages(library(vioplot))
```

Wybrane algorytmy

Poszukiwanie przypadkowe (Pure Random Search, PRS)

Losujemy po kolei zadaną z góry liczbę punktów z rozkładem jednostajnym w zadanej dziedzinie. Dziedziną jest kostką wielowymiarową, więc będziemy losować kolejno współrzędne poszczególnych punktów według odpowiedniego jednowymiarowego rozkładu jednostajnego. Każdy wylosowany punkt porównujemy z aktualnie zapamiętanym minimum i jeśli wartość minimalizowanej funkcji w tym punkcie jest mniejsza, to ten punkt zapamiętujemy jako aktualny punkt minimalny. Wartość funkcji w ostatnim zapamiętanym punkcie stanowi wynik algorytmu.

Implementacja

```
prs_algo <- function(fn, budget) {
  best_min <- Inf
  for (i in 1:budget) {
    args <- runif(
      n = getNumberOfParameters(fn),
      min = getLowerBoxConstraints(fn),
      max = getUpperBoxConstraints(fn)
    )
    best_min <- min(best_min, fn(args))
  }
  return(best_min)
}
```

Algorytm genetyczny (GA)

Wykorzystamy implementację algortmu z biblioteki `ecr` dostępnego w CRAN. Wynikiem będzie najmniejsza znaleziona wartość funkcji.

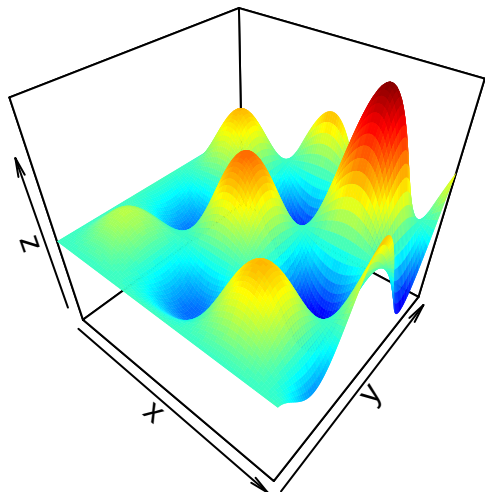
Implementacja

```
ga_algo <- function(fn, budget) {  
  minimization <- ecr(  
    fitness.fun = fn,  
    minimize = TRUE,  
    representation = "float",  
    n.dim = getNumberOfParameters(fn),  
    survival.strategy = "plus",  
    lower = getLowerBoxConstraints(fn),  
    upper = getUpperBoxConstraints(fn),  
    mu = 50,  
    lambda = 2,  
    mutator = setup(  
      mutUniform,  
      lower = getLowerBoxConstraints(fn),  
      upper = getUpperBoxConstraints(fn)  
    ),  
    terminators = list(stopOnEvals(budget)),  
    log.pop = FALSE  
  )  
  return(minimization$best.y)  
}
```

Wybrane funkcje testowe

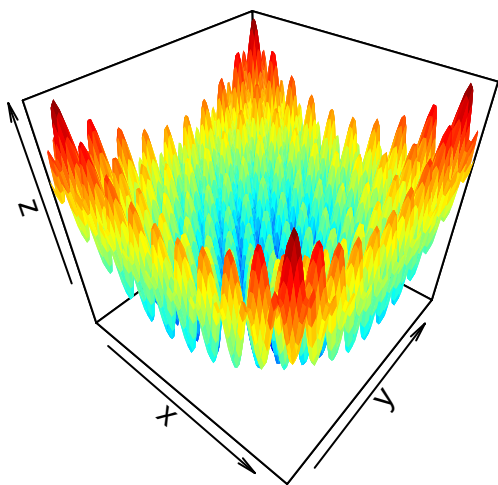
Funkcja Alpine02 dla 2 wymiarów

```
plot3D(makeAlpine02Function(dimensions = 2), colkey = FALSE)
```



Funkcja Rastrigina dla 2 wymiarów

```
plot3D(makeRastriginFunction(dimensions = 2), colkey = FALSE)
```



Rozkład wartości minimalnych

```
budget <- 1000  
color_ga <- "darkseagreen1"  
color_prs <- "dodgerblue4"
```

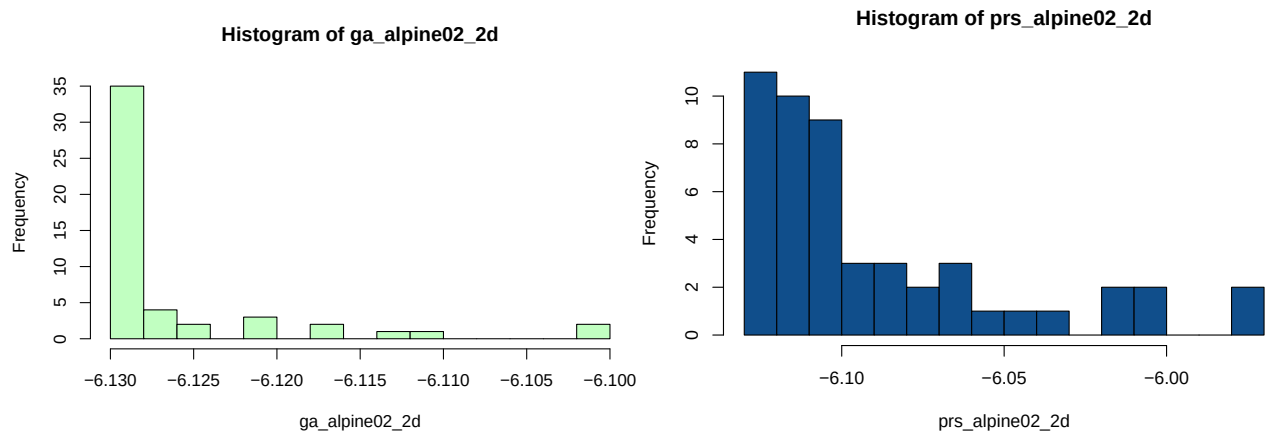
Funkcja Alpine02 dla 2 wymiarów

```
ga_alpine02_2d <- replicate(50, ga_algo(makeAlpine02Function(dimensions = 2), budget))  
prs_alpine02_2d <- replicate(50, prs_algo(makeAlpine02Function(dimensions = 2), budget))
```

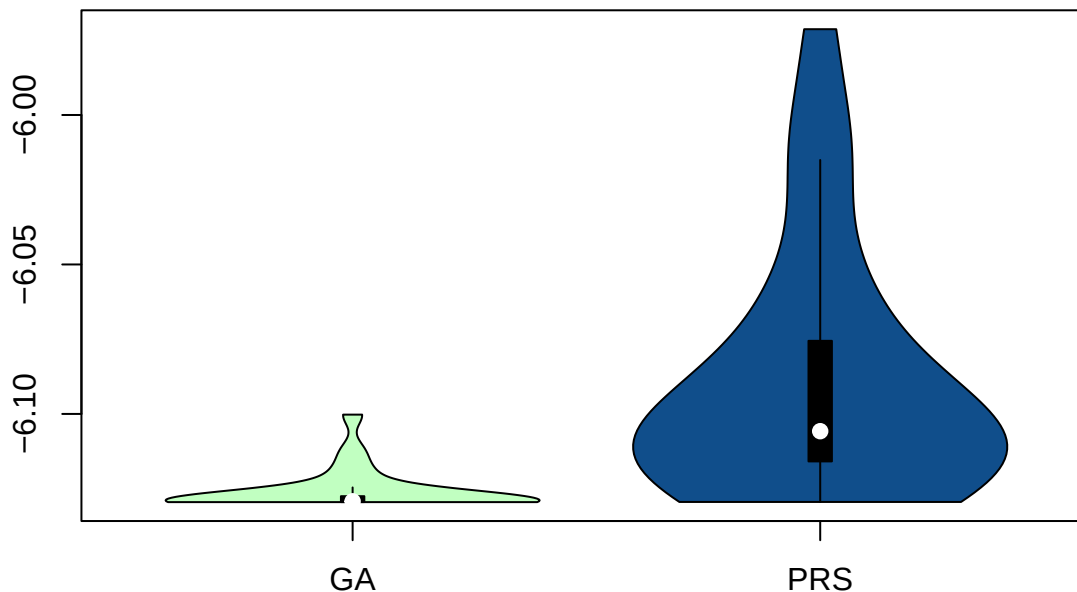
```
print(paste("Wartości średnie:", mean(ga_alpine02_2d), mean(prs_alpine02_2d)))
```

```
[1] "Wartości średnie: -6.12610129088278 -6.08926640111989"
```

```
hist(ga_alpine02_2d, breaks = 20, col = color_ga)  
hist(prs_alpine02_2d, breaks = 20, col = color_prs)
```



```
vioplot(ga_alpine02_2d, prs_alpine02_2d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



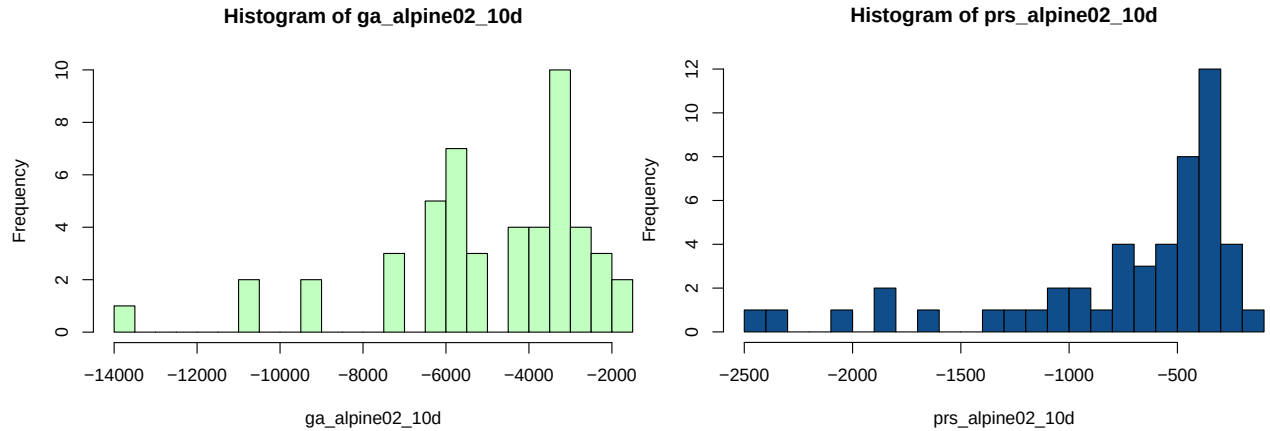
Funkcja Alpine02 dla 10 wymiarów

```
ga_alpine02_10d <- replicate(50, ga_algo(makeAlpine02Function(dimensions = 10), budget))
prs_alpine02_10d <- replicate(50, prs_algo(makeAlpine02Function(dimensions = 10), budget))
```

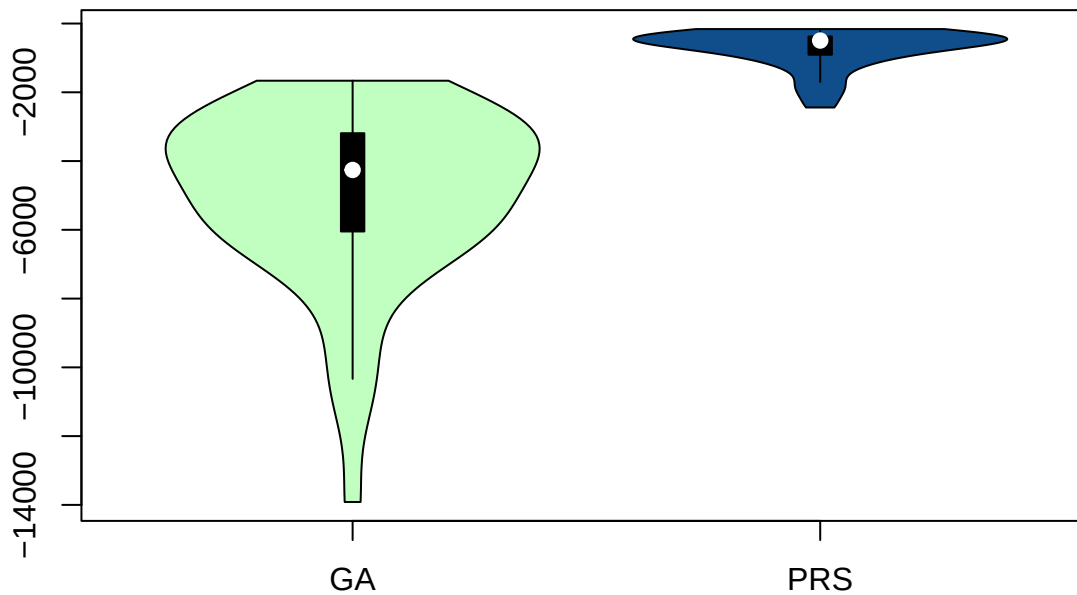
```
print(paste("Wartości średnie:", mean(ga_alpine02_10d), mean(prs_alpine02_10d)))
```

```
[1] "Wartości średnie: -4976.57476511698 -739.645874004652"
```

```
hist(ga_alpine02_10d, breaks = 20, col = color_ga)
hist(prs_alpine02_10d, breaks = 20, col = color_prs)
```



```
vioplot(ga_alpine02_10d, prs_alpine02_10d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



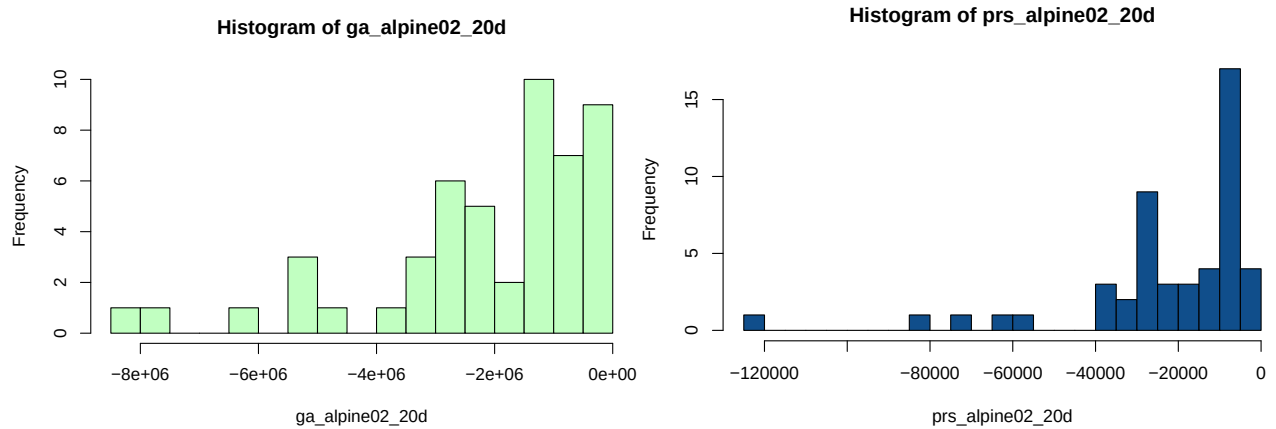
Funkcja Alpine02 dla 20 wymiarów

```
ga_alpine02_20d <- replicate(50, ga_algo(makeAlpine02Function(dimensions = 20), budget))
prs_alpine02_20d <- replicate(50, prs_algo(makeAlpine02Function(dimensions = 20), budget))
```

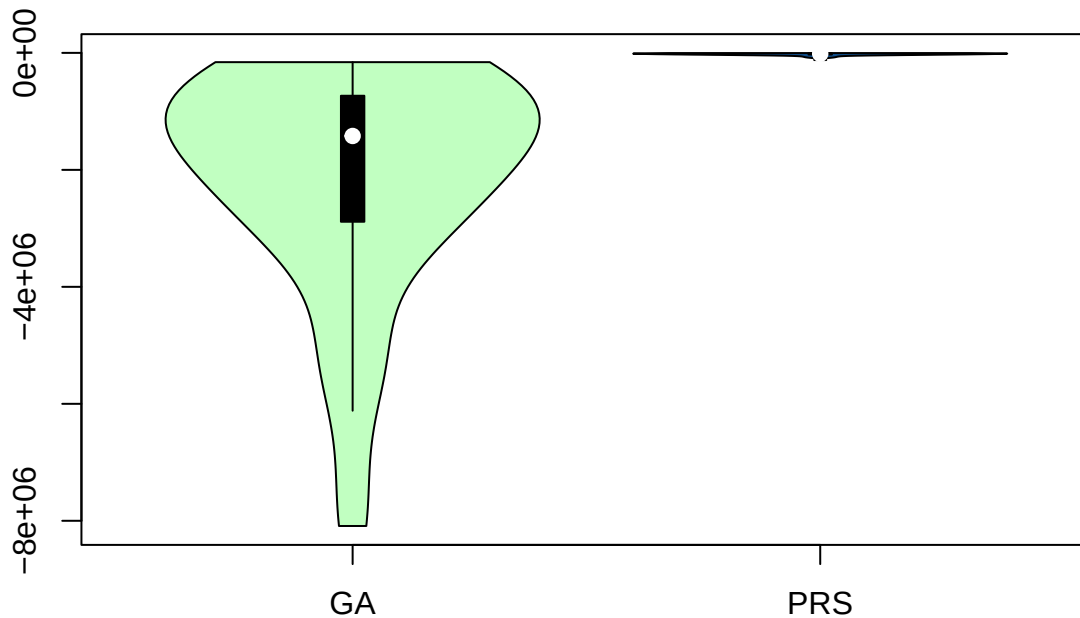
```
print(paste("Wartości średnie:", mean(ga_alpine02_20d), mean(prs_alpine02_20d)))
```

```
[1] "Wartości średnie: -2140459.67188455 -22830.9145213147"
```

```
hist(ga_alpine02_20d, breaks = 20, col = color_ga)
hist(prs_alpine02_20d, breaks = 20, col = color_prs)
```



```
vioplot(ga_alpine02_20d, prs_alpine02_20d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



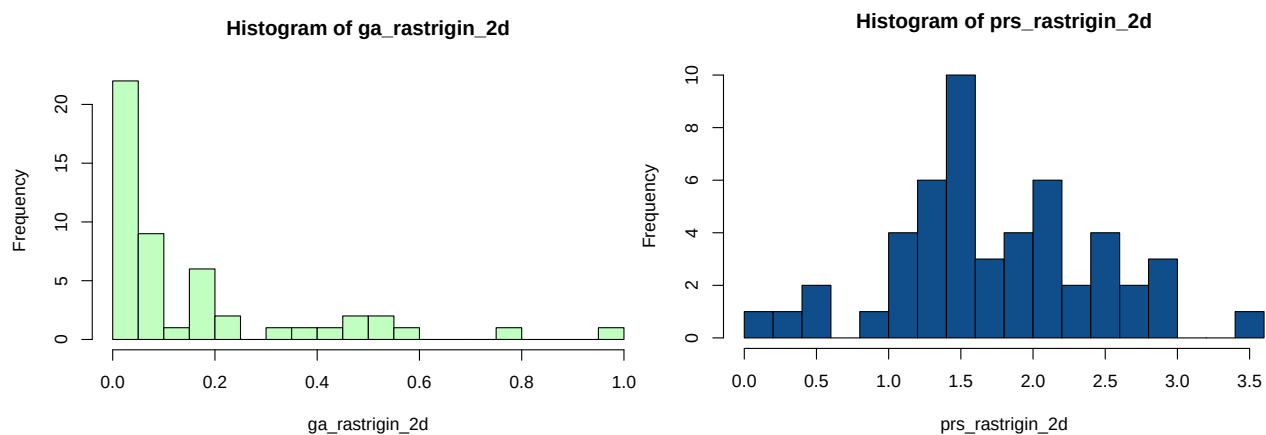
Funkcja Rastrigina dla 2 wymiarów

```
ga_rastrigin_2d <- replicate(50, ga_algo(makeRastriginFunction(dimensions = 2), budget))
prs_rastrigin_2d <- replicate(50, prs_algo(makeRastriginFunction(dimensions = 2), budget))

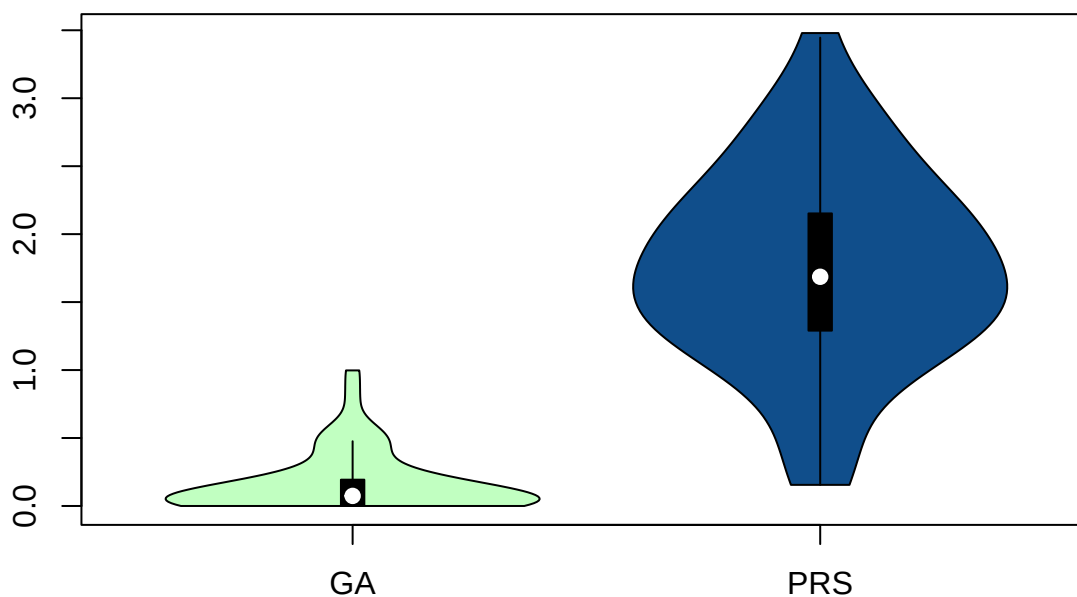
print(paste("Wartości średnie:", mean(ga_rastrigin_2d), mean(prs_rastrigin_2d)))
```

```
[1] "Wartości średnie: 0.161882086312047 1.75060388414344"
```

```
hist(ga_rastrigin_2d, breaks = 20, col = color_ga)
hist(prs_rastrigin_2d, breaks = 20, col = color_prs)
```



```
vioplot(ga_rastrigin_2d, prs_rastrigin_2d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



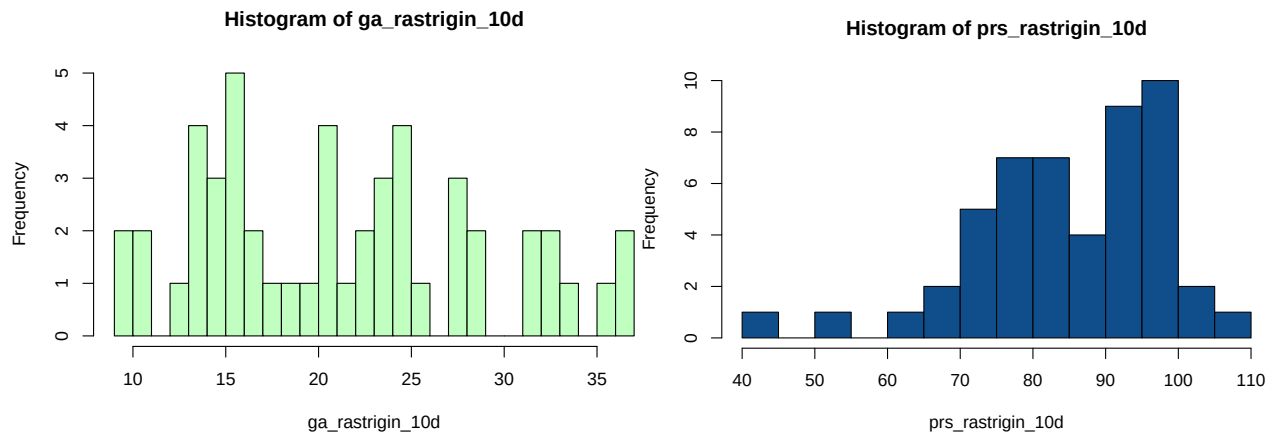
Funkcja Rastrigina dla 10 wymiarów

```
ga_rastrigin_10d <- replicate(50, ga_algo(makeRastriginFunction(dimensions = 10), budget))  
prs_rastrigin_10d <- replicate(50, prs_algo(makeRastriginFunction(dimensions = 10), budget))
```

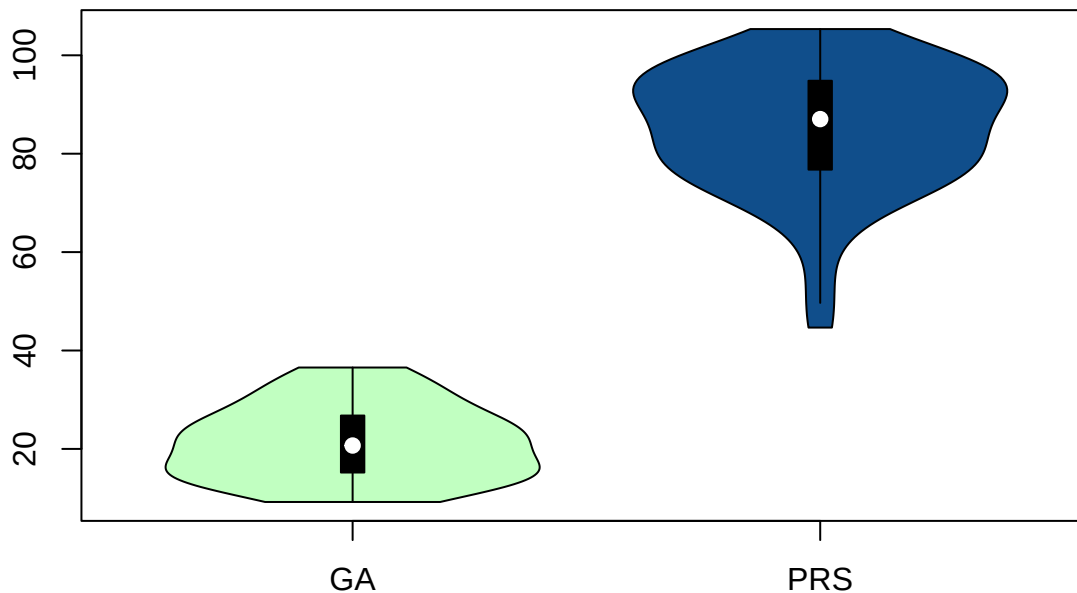
```
print(paste("Wartości średnie:", mean(ga_rastrigin_10d), mean(prs_rastrigin_10d)))
```

```
[1] "Wartości średnie: 21.3379772556348 85.1776713666971"
```

```
hist(ga_rastrigin_10d, breaks = 20, col = color_ga)  
hist(prs_rastrigin_10d, breaks = 20, col = color_prs)
```



```
vioplot(ga_rastrigin_10d, prs_rastrigin_10d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



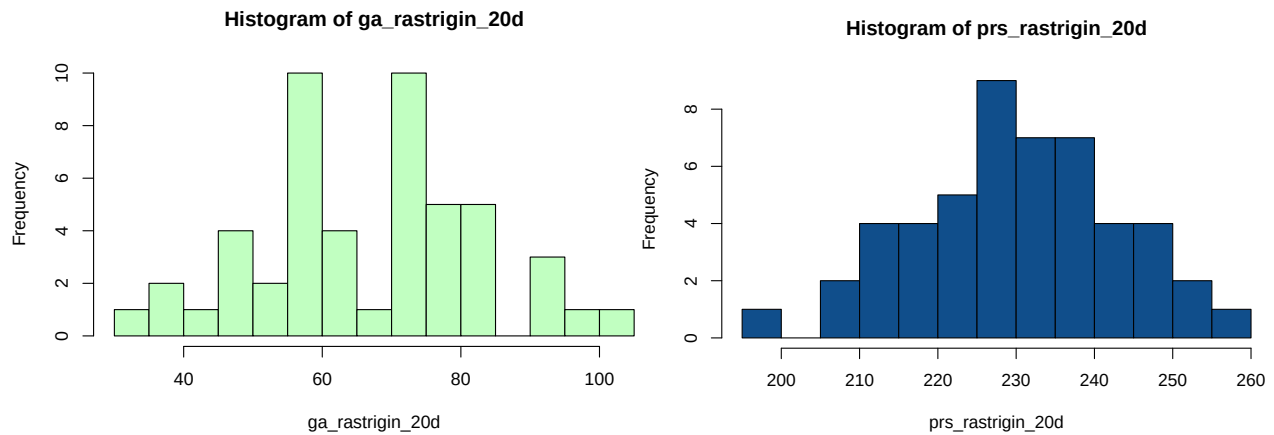
Funkcja Rastrigina dla 20 wymiarów

```
ga_rastrigin_20d <- replicate(50, ga_algo(makeRastriginFunction(dimensions = 20), budget))
prs_rastrigin_20d <- replicate(50, prs_algo(makeRastriginFunction(dimensions = 20), budget))
```

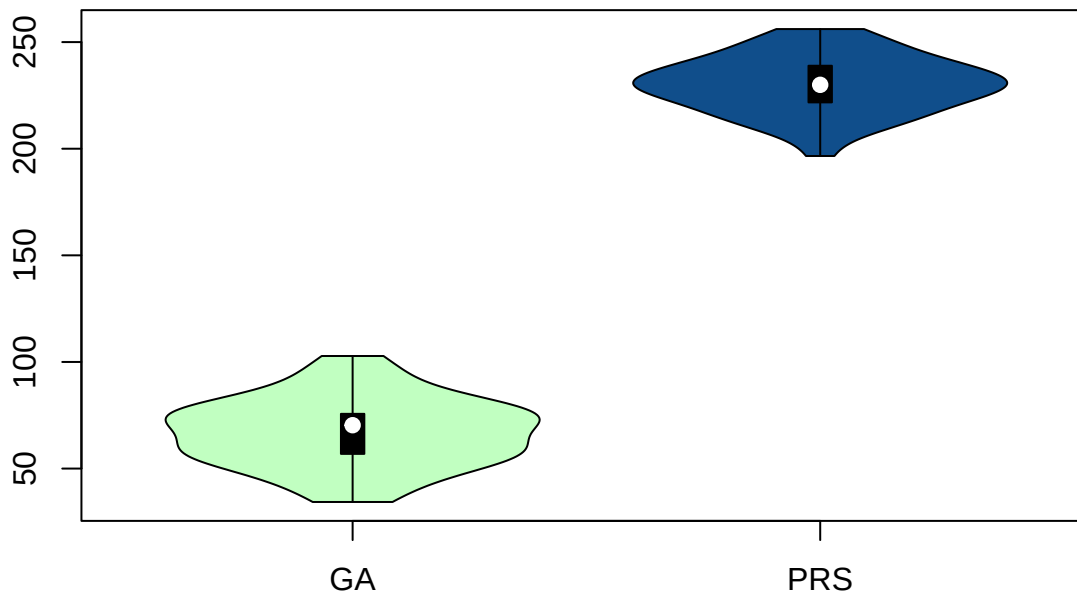
```
print(paste("Wartości średnie:", mean(ga_rastrigin_20d), mean(prs_rastrigin_20d)))
```

```
[1] "Wartości średnie: 67.0654903135273 230.146138351611"
```

```
hist(ga_rastrigin_20d, breaks = 20, col = color_ga)
hist(prs_rastrigin_20d, breaks = 20, col = color_prs)
```



```
vioplot(ga_rastrigin_20d, prs_rastrigin_20d, names = c("GA", "PRS"), col = c(color_ga, color_prs))
```



Istotności statystyczna różnicy między średnimi wynikami obu algorytmów

Wykorzystamy t-test Studenta do sprawdzenia czy różnica między średnimi wynikami obu algorytmów jest istotna statystycznie. Funkcja `t.test()` domyślnie wybiera 95 poziom ufności.

Hipoteza zerowa H_0 : średnie wyniki obu algorytmów są równe. Hipoteza alternatywna H_1 : średnie wyniki obu algorytmów są różne.

Funkcja Alpine02 dla 2 wymiarów

```
t.test(ga_alpine02_2d, prs_alpine02_2d)
```

Welch Two Sample t-test

```
data: ga_alpine02_2d and prs_alpine02_2d
t = -6.2207, df = 51.716, p-value = 8.784e-08
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.04871841 -0.02495137
sample estimates:
mean of x mean of y
-6.126101 -6.089266
```

Funkcja Alpine02 dla 10 wymiarów

```
t.test(ga_alpine02_10d, prs_alpine02_10d)
```

Welch Two Sample t-test

```
data: ga_alpine02_10d and prs_alpine02_10d
t = -11.567, df = 53.823, p-value = 3.237e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -4971.344 -3502.513
sample estimates:
mean of x mean of y
-4976.5748 -739.6459
```

Funkcja Alpine02 dla 20 wymiarów

```
t.test(ga_alpine02_20d, prs_alpine02_20d)
```

Welch Two Sample t-test

```
data: ga_alpine02_20d and prs_alpine02_20d
t = -7.8124, df = 49.014, p-value = 3.695e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -2662337 -1572921
sample estimates:
```

```
mean of x    mean of y
-2140459.67  -22830.91
```

Funkcja Rastrigina dla 2 wymiarów

```
t.test(ga_rastrigin_2d, prs_rastrigin_2d)
```

Welch Two Sample t-test

```
data:  ga_rastrigin_2d and prs_rastrigin_2d
t = -14.934, df = 58.274, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -1.801645 -1.375799
sample estimates:
mean of x mean of y
0.1618821 1.7506039
```

Funkcja Rastrigina dla 10 wymiarów

```
t.test(ga_rastrigin_10d, prs_rastrigin_10d)
```

Welch Two Sample t-test

```
data:  ga_rastrigin_10d and prs_rastrigin_10d
t = -30.729, df = 80.044, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -67.97402 -59.70537
sample estimates:
mean of x mean of y
21.33798 85.17767
```

Funkcja Rastrigina dla 20 wymiarów

```
t.test(ga_rastrigin_20d, prs_rastrigin_20d)
```

Welch Two Sample t-test

```
data:  ga_rastrigin_20d and prs_rastrigin_20d
t = -55.511, df = 94.501, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -168.9133 -157.2480
sample estimates:
mean of x mean of y
67.06549 230.14614
```